CSCE 410

P2

Kate Serem

UIN: 521009750

1. Submission Information

URL to repository

kateserem/521009750-p1-p2

Late-day tokens?

None

2. Resources Used

Resources Used:

ChatGPT

How were they useful?

- Helped me fully understand the assignment requirements and cleared up questions when I got confused about what the assignment was asking.
- It helped me fix issues I ran into, such as troubleshooting errors when building the program and debugging my code when I couldn't find the root cause.
- Helped guide me when I wasn't sure how to proceed with the assignment.
- Helped me determine additional safeguards to prevent conflicts between processes and queues, including locks, tracking whether a process is already in a queue, duplicate entries, and invalid processes.
- Assisted with debugging my round-robin scheduling test when there were multiple processes in the same priority queue.
- Assisted in building the test for aging to prevent starvation.

Estimated time resource saved:

8 Hours

Without assistance, how long would it take to complete this assignment?

20 Hours

3. Queue Design and Implementation

Priority Level Configuration

I defined NB_PRIORITY_LEVELS 3 in kernel/param.h since the file stores the settings for xv6. Defining it here, alongside the other settings used for xv6, made the most sense for consistency. I chose three priority levels: high, medium, and low.

Process Priorities

I added the priority variable priority to keep track of each process's priority level ranging from level 0 to 2, with 0 being the highest priority, 1 medium, and 2 lowest. I updated allocproc() to give the default priority level of 1 when a new process is created, kfork() to let a child process inherient their parents priority level, and lastly freeproc() for resetting the priority level to 1 when a process is freed.

System Calls: setpriority() and getpriority()

I implemented two system calls to let a process get or change its own priority. setpriority() checks if the priority is valid (0-2), returning -1 if invalid. If the priority is valid, the process calls yield() to give up the CPU before returning. If the priority is the same, it returns 0 without yielding since no change is needed. getpriority() returns the current process's priority.

Additional Safeguards

I added an in_queue variable to each process to keep track of whether it's already in the ready queue. I modified enqueue() to prevent duplicates, ensure only RUNNABLE processes are added to the queue, and check that the queue is not at the maximum number of processes. If a queue is at its max, then panic() stops the kernel and prints an error. Additionally, I initialized and reset in_queue when a process is allocated or freed. Lastly, each queue has its own lock to prevent conflicting changes.

Queue Design

I created an array ready_queues in kernel/proc.c to store the priority queues. Each queue owns an array of process pointers with a maximum size of NPROC, a count variable for the total number of processes in the queue, and a lock. count is initialized to zero, and each queue's lock is initialized in procint().

I modified the scheduler() function to use the priority queues instead of searching through the process table. The scheduler looks at the highest priority queue 0 to the lowest priority queue 2. dequeue() is used to remove the first process from the priority queue, then marks it as no longer being in the queue. If the process is runnable, then the state of the process is changed from runnable to RUNNING following the scheduler

switching to that process. After the process gives back control to the scheduler, the scheduler checks the priority queues.

To prevent starvation, I implemented aging to increase the priority of the process after its watited too long in the queue. Each process has a variable queue_time that stores the tick count when it entered the queue. The scheduler calls age_processes() to check if there are processes that have waited more than 99 ticks, moving them up one priority queue. If a process has waited too long, then remove_from_queue removes it from its current queue, increases its priority by one level, and enqueue() adds it to the new priority queue. queue_time is reset when added to the new priority queue. I later added a print statement inside age_processes() to display when a process has waited long enough to be promoted to a higher priority queue. It also prints its PID, its previous priority level, and its promoted priority level.

Unit Tests

I tested my multilevel priority scheduling using user/prioritytest.c to check if a process's priority changes, round-robin scheduling works when there are multiple processes in the same priority queue level, scheduling for processes with different priority levels, and aging. The tests and their outputs are shown in Section 5 and Section 6.

Added:

user/prioritytest.c

Modified:

Makefile
kernel/defs.h
kernel/param.h
kernel/proc.c
kernel/proc.h
kernel/syscall.c
kernel/syscall.h
kernel/sysproc.c
user/user.h
user/usys.pl

4. Process Lifecycle

Non-Trivial (significant) for Handling Process Lifecycle Events

- Assigning the default priority 1 when a new process is created.
- A child process inherits its parent's priority after forking.

- When a process becomes RUNNABLE, add it to the queue that matches its priority.
- Ensuring the scheduler checks the queue starting from the highest priority level 0 to the lowest priority 2.
- Clear the last pointer from a queue when a process is removed.
- Locking a process/queue to prevent conflicts.
- Resetting the priority level back to default 1 when a process is freed in freeproc() to prevent its previous priority from being reused
- For round-robin, ensure a process that finished its turn is added to the back of the queue.
- After a certain amount of time, move a process up a priority queue level to prevent starvation.

Bugs Encountered

When implementing the priority queues, I ran into a compilation error since the function dequeue() was created but was not being used. This occurred because I had not connected the function to the scheduler, resulting in the inability to compile the program. To fix this, i added __attribute__((unused)) to the function declaration so my code would compile. In the future, I removed it when the scheduler used it.

Minor compilation errors occurred multiple times throughout the implementation of the multilevel priority scheduler. I got errors from missing function declarations, missing header files, and typos. I also forgot to add the extern declarations for the sys calls and added sys_setpriority() and sys_getpriority() in kernel/syscall.c.

I ran into issues with processes not being removed from the priority queues properly. This occurred because the last pointer was not removed properly when a process was removed from the queue. To fix this, I updated the remove_from_queue function to remove the last pointer after removing a process by setting that last pointer to zero.

When building the round-robin scheduling, I noticed the output wasn't giving each process a turn in the order expected. This occurred because the scheduler was not given enough time to switch between processes. To fix this, I added a counting loop so each process runs longer, so the result accurately shows the processes running.

When implementing aging to prevent starvation, I ran into issues with my low-priority process not being promoted to a higher priority after 100 ticks or more.

5. Initial Test Coverage

A strategy for an initial test was making sure that the updated scheduler() would successfully boot xv6 and run basic commands such as ls and echo. This revealed whether the modified functions for ready queues and scheduler did not prevent processes from running. I used the file

kernel/proc.c and ran make CPUS=1 qemu to compile and launch xv6. After the kernel launched successfully, I ran the commands ls and echo hello to test the basic process execution. Both commands worked as expected and produced the correct output.

After implementing getpriority() and setpriority() system calls, I created a new file called user/prioritytest.c and added it to the Makefile for xv6 to run it. I used make CPUS=1 qemu to compile and launch xv6, then ran prioritytest. In prioritytest.c, it calls getpriority() to check the default priority, then changes the priority to 0 and then to 2. Additionally, it tests for an invalid priority of -1 to ensure it successfully rejects the invalid priority without changing the current priority.

prioritytest.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  //check default priority
  printf("Default priority: %d\n", getpriority());

  //change to the highest priority
  printf("Set priority to 0: %d\n", setpriority(0));
  printf("Current priority: %d\n", getpriority());

  //change to the lowest priority
  printf("Set priority to 2: %d\n", setpriority(2));
  printf("Current priority: %d\n", getpriority());

  //test invalid priority level
  printf("Set priority to -1: %d\n", setpriority(-1));
  printf("Current priority: %d\n", getpriority());

  exit(0);
}
```

Output:

```
xv6 kernel is booting

init: starting sh
$ prioritytest
Default priority: 1
Set priority to 0: 0
Current priority: 0
Set priority to 2: 0
Current priority: 2
Set priority to -1: -1
Current priority: 2
$
```

This output reveals that the default priority is 1, setpriority() successfully changed the priority to 0 and 2, and getpriority() returns the updated priority after each change. Setting the priority to -1 was rejected as expected.

6. Increased Test Coverage

After testing the syscalls setpriority() and getpriority(), I checked if a child process inherits a parent's priority when forked. I updated user/prioritytest.c to set the parent's priority to 2 before calling fork().

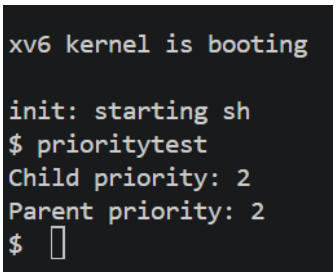
prioritytest.c

```
//test if a child inherits parent priority level
setpriority(2);

int pid = fork();

if (pid < 0) {
printf("Fork failed\n");
}
else if (pid == 0) {
//child process
printf("Child priority: %d\n", getpriority());
exit(0);
}
else {
//parent process
wait(0);
printf("Parent priority: %d\n", getpriority());
 }
```

Output



The output reveals that the child did inherit the priority level of its parent.

When there are multiple processes in the same priority queue, round-robin scheduling is used to handle when each should run. I updated user/prioritytest.c to create three child processes using fork() with the priority level 1. I then ran a counting loop to keep the CPU busy long enough for the scheduler to switch between processes. Each child then prints a message when they run so I can visually see in which order the three child processes ran.

prioritytest.c

```
        //creating three child processes
        for (int i = 0; i < 3; i++) {
          int pid = fork();

          if (pid < 0) {
            printf("Fork failed\n");
            exit(1);
```

```
    }
    if (pid == 0) {
      //set to default priority level 1
      setpriority(1);

      // Keep the child busy so the scheduler can switch processes
      for (volatile int j = 0; j < 100000000; j++) {
        if (j % 20000000 == 0) {
          printf("Child %d is running\n", i);
        }
      }
      exit(0);
    }
  }
  //parent waits for all three children to finish
  for (int i = 0; i < 3; i++) {
    wait(0);
  }
  printf("All children finished\n");
  exit(0);
```

Output

```
xv6 kernel is booting

init: starting sh
$ prioritytest
Child 0 is running
Child 1 is running
Child 2 is running
Child 0 is running
Child 1 is running
Child 2 is running
Child 1 is running
Child 2 is running
Child 0 is running
Child 1 is running
Child 2 is running
Child 0 is running
Child 1 is running
Child 2 is running
Child 0 is running
All children finished
$
```

The output reveals that when a queue has multiple processes in the same priority queue level, round-robin scheduling is working by giving each process a turn to run.

To test the scheduling with different priority levels, I updated user/prioritytest.c to create three child processes with fork(), assigning each a different priority level. I used a counting loop to keep the CPU busy and give the scheduler enough time to switch between processes.

prioritytest.c

```
//Test when processes have different priority levels
for (int i = 0; i < 3; i++) {
  int pid = fork();

  if (pid < 0) {
    printf("Fork failed\n");
```

```
      exit(1);
    }

    if (pid == 0) {
      //setting a different priority level for each child process
      setpriority(i);

      printf("Child %d has priority %d\n", i, getpriority());

      //Keeping the process busy to give scheduler enough time to switch between
processes
      for (volatile int j = 0; j < 100000000; j++) {
      }

      printf("Child %d finished\n", i);
      exit(0);
    }
  }

  //wait for all three child processes to finish
  for (int i = 0; i < 3; i++) {
    wait(0);
  }

  printf("Priority scheduling test finished\n");
```

Output

```
xv6 kernel is booting

init: starting sh
$ prioritytest
Child 0 has priority 0
Child 0 finished
Child 1 has priority 1
Child 1 finished
Child 2 has priority 2
Child 2 finished
Priority scheduling test finished
$
```

The output shows that the highest priority level finishes first, then priority level 1, and lastly priority level 2, as expected.

To test aging and starvation prevention, I updated user/prioritytest.c to create two child processes with different priority levels. I created the low-priority child with priority level 2 and the high-priority child with priority 0. I used uptime() to keep the high-priority child busy for 500 ticks, allowing the low-priority child enough time to wait in its queue. In kernel/proc.c, I updated the age_process() function to show when a process had waited too long to be run and to show its promotion to a higher-level priority queue.

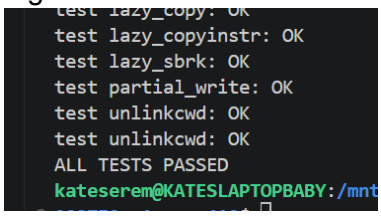
Aging test code: user/prioritytest.c (located in GitHub repository)

Output:

```
xv6 kernel is booting

init: starting sh
$ prioritytest
High-priority child started at priority 0
[AGING] PID 4 waited 100 ticks and needs to age
[AGING] PID 4 aged from priority 2 to priority 1
[AGING] PID 4 waited 100 ticks and needs to age
[AGING] PID 4 aged from priority 1 to priority 0
low-priority child first running at priority 0
low-priority child finished at priority 0
High-priority child finished at priority 0
$
```

I additionally ran the provided xv6 test to make sure the changes to xv6 did not break its functionality. I ran CPUS=1 python3 test-xv6.py -q usertests, and the output showed “ALL TESTS PASSD.” verifying that the existing xv6 passed after implementing multilevel queue scheduling.



Problems Indeitified/Debugged/Had to Fix

When I ran my test, the output showed that each process was not given a turn. This was because the scheduler was not given enough time to switch between processes. Therefore, I added a counting loop to keep the CPU busy and print messages during the loop, as suggested by ChatGPT.

prioritytest.c

```
//TEST AGING TO PREVENT STARVATION

//setting the priority to 2 for when we create the low priority child
setpriority(2);

int low_pid = fork();

if (low_pid < 0) {
  printf("Fork failed\n");
  exit(1);
}
if (low_pid == 0) {
  printf("low-priority child first running at priority %d\n",getpriority());

  //keep the low priority child busy
  int start = uptime();

  while (uptime() - start < 200) {
    //keep the CPU busy for 200 ticks after the low priority child gets to run
  }
```

```
  printf("low-priority child finished at priority %d\n",  getpriority());
  exit(0);
}
//parent gets priority 0
setpriority(0);

//create the high priority child
int high_pid = fork();

if (high_pid < 0) {
  printf("Fork failed\n");
  exit(1);
}
if (high_pid == 0) {
  printf("High-priority child started at priority %d\n", getpriority());

  //keep priority 0 busy long enough for aging to occur in the low-priority child
  int start = uptime();

  while (uptime() - start < 500) {
    //keep the CPU busy for 500 ticks so low priority child can age
  }
  printf("High-priority child finished at priority %d\n", getpriority());
  exit(0);
}
//waiting for both children to finish
wait(0);
wait(0);
exit(0);
```

Output

```
xv6 kernel is booting

init: starting sh
$ prioritytest
High-priority child started at priority 0
[AGING] PID 4 waited 100 ticks and needs to age
[AGING] PID 4 aged from priority 2 to priority 1
[AGING] PID 4 waited 100 ticks and needs to age
[AGING] PID 4 aged from priority 1 to priority 0
low-priority child first running at priority 0
low-priority child finished at priority 0
High-priority child finished at priority 0
$
```

The output shows that the low-priority child waited 100 ticks before being promoted from priority 2 to priority 1, then priority 0. The low-priority child was able to finish before the high-priority child, showing that aging worked.